

CERTIFICACIÓN UNIVERSITARIA



Data end

SQL



**WE WANT
SKILLS!**

mundosE



SQL (*structured query language*)

El SQL o **lenguaje estructurado de consultas**, define las reglas de comunicación entre bases de datos estructuradas y el ambiente externo. Entendemos por **ambiente externo** cualquier herramienta de *software* que desee interactuar con una base de datos relacionales (también existen las bases de datos no estructuradas).

SQL, en algunos casos, puede llegar a definirse como un lenguaje de programación, aunque no tenga todas las características de ello. En realidad, SQL tiene disponible algunas opciones (que veremos más adelante) y que nos permiten desarrollar procesos programados.

SQL trabaja con comandos o sentencias. Mediante la definición de su sintaxis podemos ejecutar todas las operaciones posibles.

¿Qué es una base de datos relacionales?

Empecemos por definir qué es una base de datos. De acuerdo con el sitio web de Oracle (principal motor de base de datos SQL), una **base de datos** es una recopilación organizada de información o **datos** estructurados, **que** normalmente se almacena de forma electrónica en un sistema informático. Normalmente, una **base de datos** está controlada por un sistema de gestión de **bases de datos** (DBMS).

Los Motores de Bases de Datos, son programas específicos, dedicados a servir de intermediarios entre las bases de datos y las aplicaciones que las utilizan como función principal, cada uno de estos cumple una tarea específica, que van desde crear la Base de Datos hasta



administrar el uso y acceso a esta. (Infase, 2017, <https://infase.net/motores-de-bases-de-datos/>)

Es a través del cliente DBMS que podemos ejecutar los comandos o sentencias SQL sobre una base de datos.

Conceptos claves de una base de datos

Cuando almacenamos datos, realizamos el modelado de estos, de forma tal que identifiquemos qué valores forman parte de una entidad de información. Una entidad de información es un conjunto de valores que permiten definir un objeto, ente o persona. Entonces, cuando trabajamos con bases de datos, modelamos las entidades de información y sus relaciones, por ello hablamos de bases de datos relacionales.

La unidad mínima de datos que podemos manejar es el **campo**. Un campo es cada uno de los valores que identificaremos. Cada campo tiene un nombre que permite conocer la naturaleza del valor almacenado.

El conjunto de campos que permiten definir una entidad de información se denomina **tabla**. Entonces, una **tabla** será uno o más campos que, juntos, representan un modelo.

Luego, podemos **relacionar** estas tablas a partir de alguno de los campos que contienen. De esta manera, tenemos modelos de información mucho más elaborados.

Cada representación de modelo se guarda en la tabla como una entrada o línea. Esto se denomina **registro**. Entonces, una tabla puede contener 0 a n registros.



¿Qué procesos o acciones podemos realizar con SQL?

Dividiremos los procesos en las siguientes categorías:

- DDL (*data definition language*): estructura de base de datos.
- DML (*data manipulation language*): gestión de registros.
- DQL (*data query language*): consulta de registros.
- Procesos programados.

DDL: estructura de base de datos

Como indicamos anteriormente, una base de datos se organiza de manera tal que los datos sean estructurados y relacionados. Con esta premisa, entenderemos cómo el SQL nos ayuda a realizar una serie de operaciones sobre una base de datos.

Creación de base de datos

El primer paso para definir y estructurar nuestros datos es la creación de una base de datos. Se realiza desde el cliente DBMS, al cual nos conectamos mediante la autenticación de datos. Todo DBMS posee una gestión de seguridad, de forma tal que sea necesario indicar un usuario y clave para poder acceder al cliente. Para crear una base de datos basta con ejecutar el siguiente comando:

```
CREATE DATABASE nombre;
```

En el comando anterior, el *nombre* es el identificador con el cual el DBMS reconocerá nuestra base de datos. Cada DBMS, adicionalmente, puede tener otras opciones, pero en esencia todo DBMS acepta la sintaxis anterior (SQL ANSI).



Modificación de la base de datos

Las alteraciones o actualizaciones sobre la base de datos como estructura, dependerán del DBMS en que se esté trabajando. La mayoría de ellos acepta actualizar dos variables: la codificación de los caracteres o ENCODING (codificación) y la forma en que se realizan las búsquedas y orden de resultados, que llamamos COLLATE (colación).

De acuerdo a las opciones del manejador, el comando se ajustará, pero todo comando de ajuste sobre la base de datos tendrá, al menos, esta sintaxis:

```
ALTER DATABASE name <opciones>;
```

Opciones del comando ALTER DATABASE en los DBMSs mas populares:

- ALTER TABLE en MySQL. Puedes ver más en <https://dev.mysql.com/doc/refman/8.0/en/alter-database.html>.
- ALTER TABLE en Postgresql. Puedes ver más en <https://www.postgresql.org/docs/current/sql-alterdatabase.html>.
- ALTER TABLE en SQL Server. Puedes ver más en <https://learn.microsoft.com/es-es/sql/t-sql/statements/alter-database-transact-sql?view=sql-server-ver16&tabs=sqlpool>.

Borrado de base de datos

Mediante el siguiente comando podemos borrar la base de datos y todo su contenido. Es importante tener especial cuidado con este comando, pues luego de aplicado, todo lo relacionado a la base de datos será excluido sin posibilidad de recuperación.

El comando necesario para realizar esta operación es el siguiente:



```
DROP DATABASE name;
```

Ahora, entremos en el detalle de la estructura de una base de datos. Como comentamos anteriormente, una base de datos está compuesta por tablas (que, a su vez, están integradas por campos) y relaciones. Cada tabla puede tener campos, índices, clave primaria y claves foráneas.

Creación de tablas

Para crear una tabla, definimos su nombre y los campos que la componen. En este caso, es importante recordar que las tablas se componen de campos, y cada campo puede tener un tipo de dato. Los tipos de datos soportados por la mayoría de DBMS son los siguientes:

- Caracteres.
- Cadena de caracteres (hasta 512 caracteres).
- Enteros.
- Decimales.
- Textos o memo.
- Binarios.

Cada DBMS maneja nombres diferentes para los tipos de datos, así como poseen tipos de datos propios. Por ello, es importante revisar los siguientes enlaces donde podemos acceder a los tipos de datos de los DBMS más populares:

- Tipos de datos en MySQL. Ver sitio: <https://dev.mysql.com/doc/refman/8.0/en/data-types.html>.
- Tipos de datos en PostgreSQL. Ver sitio: <https://www.postgresql.org/docs/current/datatype.html>.



- Tipos de datos en SQLServer. Ver sitio: <https://learn.microsoft.com/es-es/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver16>.

Para crear una tabla, definimos su nombre y sus campos. De esta forma, un comando aceptado por la mayoría de los DBMS sería el siguiente:

```
CREATE TABLE name (  
    field1 datatype,  
    field2 datatype,  
    field3 datatype,  
    ....  
);
```

En el *script* anterior, tanto *name* (nombre de la tabla) como *field1*, *field2*, *field3*, etc. son los campos o columnas que componen las tablas. Estos identificadores deben seguir reglas como:

- No usar caracteres especiales.
- No utilizar espacios (algunos DBMS los aceptan, pero no se recomienda).
- Usar letras minúsculas.

Ejemplo de creación de tablas en los DBMSs más populares:

Crear una tabla en MySQL

```
CREATE TABLE personas (  
    id int,  
    apellidos varchar(255),  
    nombres varchar(255),  
    direccion varchar(255),  
    ciudad varchar(255)  
);
```



Crear una tabla en PostgreSQL

```
CREATE TABLE IF NOT EXISTS personas (  
    id INT PRIMARY KEY,  
    nombres VARCHAR(50) NOT NULL,  
    apellidos VARCHAR(50) NOT NULL,  
    genero CHAR(1),  
    email VARCHAR(100) UNIQUE,  
    pais_nacimiento VARCHAR(50)  
);
```

Crear una tabla en SQL Server

```
CREATE TABLE employees  
( id INT NOT NULL,  
  nombres VARCHAR(50) NOT NULL,  
  apellidos VARCHAR(50),  
  salario MONEY  
);
```

Observa que la sintaxis es bastante similar entre los sistemas, pero hay que tener cuidado con los tipos de datos, pues cada DBMS tiene sus propias reglas y su sintaxis, sobre todo en campos especializados o de trato especial.

Cómo agregar campos a tablas existentes

Luego de definir una tabla podemos ajustar sus campos, sin necesidad de crearla nuevamente. Esto significa que podemos agregar nuevos campos o, incluso, podemos modificar los existentes. Para este punto haremos uso del siguiente comando:

```
ALTER TABLE name ....
```

El comando ALTER TABLE nos permite realizar cambios sobre la tabla indicada. Nuevamente, la sintaxis puede tener ligeras variaciones de acuerdo al DBMS que se utilice, pero en esencia se mantiene su estructura. Veamos, por ejemplo, cómo agregar un campo:



```
ALTER TABLE nombre ADD fecha_nacimiento DATE;
```

El comando anterior agrega el campo `fecha_nacimiento` de tipo `DATE` (fecha) a la tabla *nombre*. Es un comando ANSI válido en los 3 manejadores que hemos mencionado en la documentación.

Cómo cambiar la definición de un campo

En este caso, usamos nuevamente el comando `ALTER TABLE`. Cada manejador implementa diferentes sintaxis para cambiar la definición de un campo:

Cambiar un campo en MySQL

```
ALTER TABLE nombre MODIFY fecha_nacimiento VARCHAR(10)
```

Cambiar un campo en PostgreSQL

```
ALTER TABLE nombre ALTER COLUMN fecha_nacimiento SET DATA TYPE  
VARCHAR(10);
```

Cambiar un campo en SQL Server

```
ALTER TABLE nombre ALTER COLUMN fecha_nacimiento VARCHAR(10);
```

Si observamos los comandos, podremos comprobar que, aunque los DBMS intentan mantener una sintaxis similar, cada uno tiene opciones y manejos diferentes, por lo tanto, te dejo acá el *link* a la documentación oficial del comando.

- ALTER TABLE en MySQL. Ver sitio:
<https://dev.mysql.com/doc/refman/8.0/en/alter-table.html>.
- ALTER TABLE en PostgreSQL. Ver sitio:
<https://www.postgresql.org/docs/current/sql-altertable.html>.



- ALTER TABLE en SQL Server. Ver sitio : <https://learn.microsoft.com/es-es/sql/t-sql/statements/alter-table-transact-sql?view=sql-server-ver16>.

Cómo borrar campos

Del mismo modo en que realizamos ajustes sobre campos, ya sea para agregar campos tiempo después de crear la tabla o para cambiar la definición de los existentes, con el comando `ALTER TABLE` también es posible borrar campos.

El comando para realizar esta operación es:

```
ALTER TABLE nombre_tabla  
DROP COLUMN nombre_campo;
```

Este comando es válido en los DBMS más conocidos del mercado, como MySQL, PostgreSQL y SQL Server.

Cómo agregar claves primarias

Empecemos por recordar o entender qué son las claves primarias. Una clave primaria es una característica que posee la tabla de una base de datos relacionales y que nos ayuda en la forma de organizar los datos al indicar que ese campo debe poseer valores únicos, es decir, cada valor solo debe estar asignado a un registro. De esta forma, garantizamos que podemos identificar cada registro unívocamente. Una clave primaria también nos ayuda a buscar y a relacionar datos entre una tabla y otra, pues a través de ella el DBMS sabe qué registro está relacionado con qué otro registro.

¿Es posible tener tablas sin claves primarias? Sí, es posible, pero es una mala práctica porque, en general, siempre cada registro posee alguna forma de identificación única. Entonces, si no la definimos, hacemos que el DBMS



realice un trabajo que impacta en el rendimiento de nuestras consultas y operaciones.

Una clave primaria puede ser creada en 2 momentos, tanto en la creación de la tabla como luego, en forma de actualización. La sintaxis para crear una clave primaria se muestra a continuación, aunque puede tener algunas opciones que pueden variar por DBMS, en general, se aplica de la siguiente forma:

1. **Al momento de crear la tabla.** En este ejemplo usamos el campo *id* como identificador de cada registro de las tablas.

```
CREATE TABLE nombre_tabla (  
    id int NOT NULL,  
    ...  
    PRIMARY KEY (id)  
);
```

2. **Al actualizar la tabla.**

```
ALTER TABLE nombre_tabla ADD CONSTRAINT pk_nombre_tabla PRIMARY KEY (id)
```

Cómo agregar claves foráneas

Una clave foránea o llave extranjera nos permite definir comportamientos o acciones a ejecutar cuando un registro tiene otros registros relacionados.

Algunas de las acciones que son soportadas por la mayoría de los DBMS son:

- Restringir la actualización del campo relacionado o el borrado de registros si existen registros relacionados mediante una clave foránea.



- Actualizar el campo relacionado en cascada o borrar registros en cascada a partir del registro principal; es decir, borrar los registros relacionados a partir del principal.
- Asignar valor *null* al campo relacionado en los registros relacionados al borrar.

¿Cómo se agrega una clave foránea? Esta sintaxis varía de acuerdo al DBMS, pero vamos a intentar unificarla, tanto en la creación como en la actualización de tablas.

1. Al crear una tabla podemos crear una clave foránea de la siguiente manera:

```
CREATE TABLE nombre_tabla (  
    id int NOT NULL,  
  
    PRIMARY KEY (id),  
    CONSTRAINT fk_tabla_relacionada FOREIGN KEY (id)  
    REFERENCES tabla_relacionada(campo_relacionado)  
);
```

2. Al actualizar la tabla podemos crear la clave foránea mediante el comando ALTER TABLE:

```
ALTER TABLE nombre_tabla  
ADD CONSTRAINT fk_tabla_relacionada  
FOREIGN KEY (id) REFERENCES tabla_relacionada(campo_relacionado);
```

A continuación, veremos las acciones que podemos ejecutar. Cabe recordar que no todos los DBMS lo soportan:

- Cláusula *ON UPDATE*:
 - *ON UPDATE RESTRICT*: el valor predeterminado. Si intenta actualizar un *id* en la tabla principal, el DBMS rechazará la operación si al menos un relacionado se vincula a la tabla principal.
 - *ON UPDATE NO ACTION*: funciona igual que RESTRINGIR.



- *ON UPDATE CASCADE*: suele ser la mejor opción. Si actualiza un *id* en un registro de la tabla principal, el DBMS lo actualizará, en consecuencia, en todas las filas de la tabla relacionada que hacen referencia a este. Cabe advertir que no se activan las relaciones en la tabla relacionada.
- *ON UPDATE SET NULL*: si actualiza un *id* en un registro de la tabla principal, el motor establecerá el *id* relacionados en la tabla relacionada en NULL (debe estar disponible ese valor en el campo relacionado).
- Cláusula ON DELETE:
 - *ON DELETE RESTRICT*: el valor predeterminado. Si intenta eliminar un *id* en la tabla principal, el motor rechazará la operación si al menos un registro relacionado existe, puede salvarle la vida.
 - *ON DELETE NO ACTION*: funciona igual que RESTRINGIR.
 - *ON DELETE CASCADE*: es peligroso. Si elimina un registro de la tabla principal, el motor también eliminará los registros relacionados. Esta acción es peligrosa, pero se puede utilizar para realizar limpiezas automáticas en tablas secundarias.
 - *ON DELETE SET NULL*: si elimina una fila de la tabla principal, los registros relacionados tendrán automáticamente la relación con NULL.

Cómo agregar índices

Los índices en las tablas son elementos que nos permiten mejorar la *performance* a la hora realizar búsquedas, además de ayudar en la creación de campos con valores únicos en cada registro, de tal forma que ese valor no se repita.

¿Como se agrega un índice? Es sencillo, podemos realizar esta operación mediante la siguiente sintaxis:

```
CREATE [UNIQUE] INDEX <nombre_indice> ON <nombre_tabla> (<campo1...>);
```



Cuando creamos un índice, podemos definir uno o más campos. De esta forma, se puede crear un conjunto de campos cuya combinación sea única mediante la palabra `UNIQUE`, pero no siendo obligatorio su uso.

Cómo borrar tablas

DML: gestión de registros

En este conjunto de operaciones, se encuentran los procesos de inserción de nuevos registros, actualización de registros y borrado de registros.

Comencemos por la inserción o ingreso de nuevos datos.

Inserción de registros

Para realizar la inserción o el ingreso de nuevos registros o filas a una tabla usamos el comando `INSERT INTO`, donde se especifican los campos o columnas y sus respectivos valores.

La sintaxis de este comando es similar en todos los DBMS populares. Sigue la anatomía de la sintaxis:

```
INSERT INTO nombre_tabla (campo1, campo2, ...)
VALUES (valorCampo1, valorCampo2, ...)
```

De esta forma, conseguimos agregar un registro en la tabla `nombre_tabla`. Es importante seguir algunas reglas:

- Campos de texto o fecha: deben especificar su valor entre comillas simples.
- Campos numéricos: pueden ir o no entre comillas.
- Hay que tener cuidado con los distintos tipos de datos, si intentamos asignar un valor de texto a un campo numérico (entero o decimal), el DBMS nos arrojará un error.
- Si tenemos campos que no aceptan nulos, significa que debemos siempre mandar un valor, hasta puede ser vacío entre comillas, pero debe enviarse un valor.



Actualización de registros

Cuando deseamos actualizar un registro, debemos tener en cuenta 2 aspectos:

- Campos a actualizar.
- Condición que deben cumplir los registros que se actualizarán.

Con las anteriores premisas, podemos mostrar la sintaxis que debe seguir el proceso de actualización:

```
UPDATE nombre_tabla SET campo1 = 'valor1', campo2 = 'valor2', ...  
-- condicion  
WHERE campoX = 'valorX' AND / OR campoY = 'valorY'
```

Otro punto interesante acerca de las condiciones es entender que cuando aplicamos la palabra `WHERE` le indicamos al DBMS que queremos filtrar los registros. Ese filtro puede ser por 1 o más campos. Cuando usamos más de un campo entran en escena los operadores lógicos, que se aplican tanto en la actualización, como en el borrado o la consulta. Más adelante, ahondaremos en esta explicación.

Borrado de registros

Es importante entender el proceso de borrar o quitar registros para no realizar operaciones que luego deriven en pérdida de datos. Cuando borramos registros o líneas, lo hacemos sobre una tabla y aplicaremos una condición, que puede ser simple o compleja, entendiendo que una condición compleja amerita el uso de los operadores lógicos, tal como en el proceso de actualización que veremos en la siguiente sección.

Sintaxis o anatomía de una operación de borrado:

```
DELETE FROM nombre_tabla  
-- condición  
WHERE campo1 = 'valor1' AND / OR campo2 = 'valor2' ...
```

Es importante prestar atención y aplicar las condiciones correctas para evitar pérdida de datos.



Operadores lógicos en SQL

Los operadores lógicos son los términos que nos permitirán definir condiciones complejas, donde podemos involucrar n campos y, mediante operadores de precedencia como los paréntesis, podremos definir prioridad de la evolución de la condición.

Operadores lógicos soportados por la mayoría de los DBMS:

- ALL: verdadero, si el conjunto completo de comparaciones es verdadero.
- AND: verdadero, si ambas expresiones booleanas son verdaderas. Ver sitio: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/and-transact-sql?view=sql-server-ver16>.
- ANY: verdadero, si cualquier miembro del conjunto de comparaciones es verdadero. Ver sitio: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/any-transact-sql?view=sql-server-ver16>.
- BETWEEN: verdadero, si el operando está dentro de un intervalo de valores. Ver sitio: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/between-transact-sql?view=sql-server-ver16>.
- EXISTS: verdadero si una subconsulta contiene cualquiera de las filas. Ver sitio: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/exists-transact-sql?view=sql-server-ver16>.
- IN: verdadero si el operando es igual a uno de la lista de expresiones. Ver sitio: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/in-transact-sql?view=sql-server-ver16>.
- LIKE: verdadero, si el operando coincide con un patrón. Ver sitio: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/like-transact-sql?view=sql-server-ver16>.
- NOT: invierte el valor de cualquier otro operador booleano. Ver enlace: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/not-transact-sql?view=sql-server-ver16>.
- OR: verdadero, si cualquiera de las dos expresiones booleanas es verdadera. Ver enlace: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/or-transact-sql?view=sql-server-ver16>.
- SOME: verdadero, si alguna de las comparaciones de un conjunto es verdadera. Ver enlace: <https://learn.microsoft.com/es-es/sql/t-sql/language-elements/some-any-transact-sql?view=sql-server-ver16>.



En la parte de consulta de registros veremos en detalle estos operadores, pero debemos entender que al momento de actualizar y borrar podemos utilizarlos, con algunas restricciones.

Operaciones de actualización y borrado con una tabla principal y tablas relacionadas

Vimos que las operaciones sobre registros se aplican sobre una tabla siempre. No obstante, tanto el comando `UPDATE` como `DELETE` pueden ser aplicados sobre una tabla principal si se consideran las condiciones de las tablas relacionadas.

Actualizar una tabla con condición relacionada a otras tablas

Esta operación varía de acuerdo al DBMS, veamos cómo funciona en MySQL.

```
UPDATE nombre_tabla
  JOIN tabla_relacionada
    SET nombre_tabla.campo = 'valor'

WHERE nombre_tabla.campo_relacionado = tabla_relacionada.campo_relacionado
```

En otros DBMS, el comando usa una sintaxis diferente, como sigue a continuación:

```
UPDATE nombre_tabla
  SET nombre_tabla.campo = 'valor'
  JOIN tabla_relacionada
WHERE nombre_tabla.campo_relacionado = tabla_relacionada.campo_relacionado
```

Cómo borrar registros de una tabla con condición relacionada a otras tablas

Esta operación varía de acuerdo al DBMS, veamos cómo funciona en MySQL

```
DELETE nombre_tabla FROM nombre_tabla
  JOIN tabla_relacionada
WHERE nombre_tabla.campo_relacionado = tabla_relacionada.campo_relacionado
```

En otros DBMS, el comando usa otra sintaxis:



```
DELETE FROM nombre_tabla
  USING tabla_relacionada
 WHERE nombre_tabla.campo_relacionado = tabla_relacionada.campo_relacionado
```

Consulta de registros

Para consultar registros aplicamos el comando `SELECT`. Este comando tiene diversas formas de usarse, empecemos por entender su anatomía básica y avanzaremos gradualmente en cuanto a complejidad y potencia de uso.

Consulta simple

Podemos iniciar el conocimiento del comando `SELECT` consultando información de una sola tabla. Se sigue la anatomía a continuación:

```
SELECT
-- CAMPOS
  tabla1.campo1, tabla1.campo2, ...
-- o podemos usar * para todos los campos
-- DESDE TABLA
  FROM tabla1
-- CONDICION
 WHERE tabla1.campo1 = 'valor1' AND / OR tabla1.campo2 = 'valor2' ...
```

En la parte de condición, podemos usar los operadores lógicos vistos anteriormente.

Consulta con tablas relacionadas

Cuando realizamos una consulta, podemos relacionar diferentes tablas, siempre que estas estén vinculadas entre sus campos. En estos casos es recomendable usar: claves foráneas e índices para mejorar el rendimiento de la consulta.



Forma de definir una consulta con tablas relacionadas:

```
SELECT
-- CAMPOS
    tabla1.campo1, tabla2.campo2, ...
-- DESDE TABLAS
    FROM tabla1, tabla2, ...
-- CONDICION
WHERE tabla1.campo1 = 'valor1' AND / OR tabla2.campo2 = 'valor2' ...
```

Consulta con tablas relacionadas mediante criterios

Para realizar consultas relacionadas tenemos disponible una palabra reservada llamada **JOIN**, que permite realizar consultas con diferentes criterios y recuperar los registros mediante criterios.

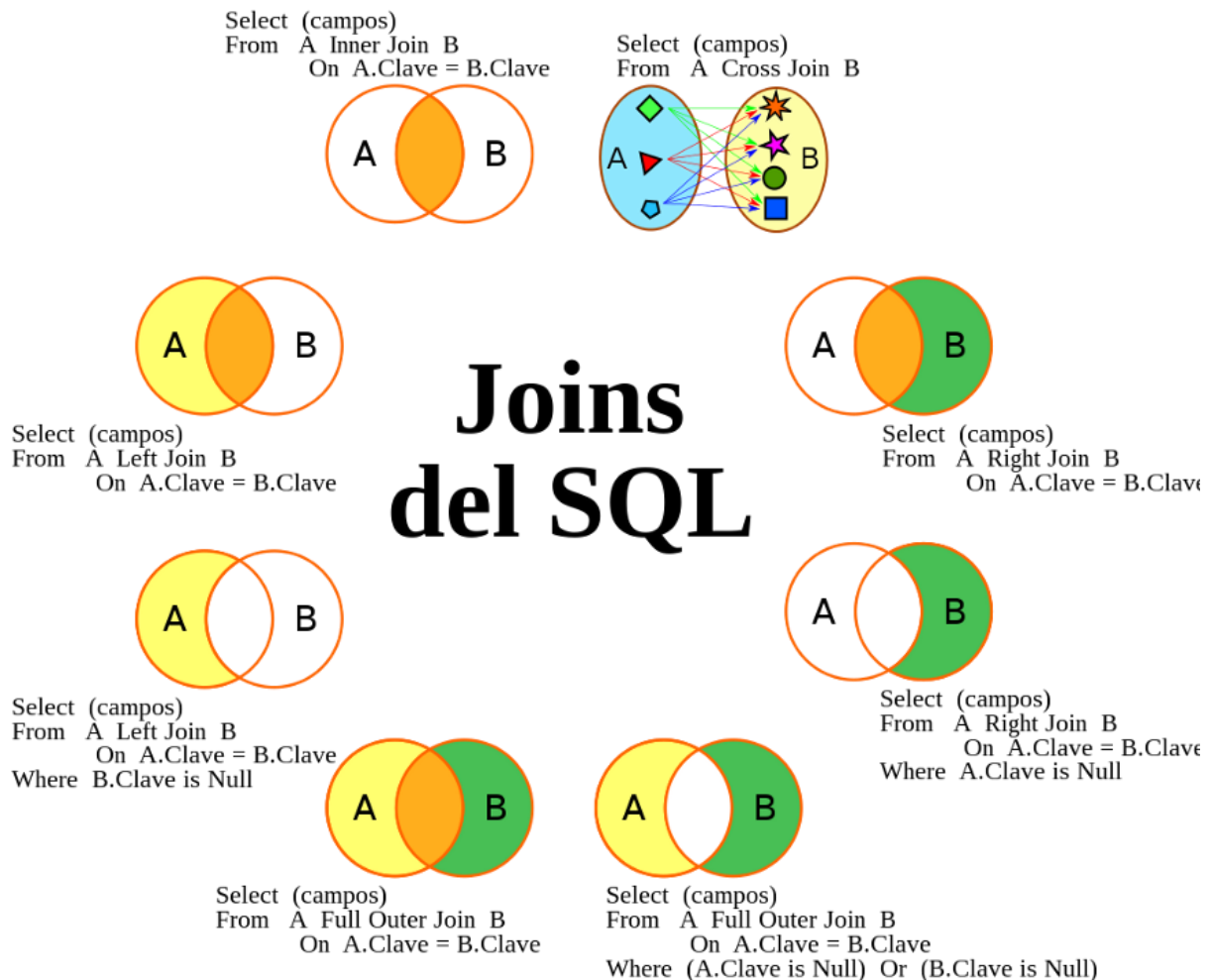
Con **JOIN** podemos definir la forma en que se relacionan las tablas, es decir, el grado de relación entre las tablas:

- Si los registros deben estar en ambas tablas. Condición de intersección.
- Toda la tabla A y lo que cumpla la condición de la tabla B.
- Toda la tabla B y lo que cumpla la condición de la tabla A.
- Tabla A, menos lo que cumpla la condición de ambas tablas.
- Tabla B, menos lo que cumpla la condición de ambas tablas.
- Tabla A y tabla B, cumplan o no la condición.
- Tabla A y tabla B, siempre que no cumplan o no la condición.

Veámoslo en forma gráfica:



Figura 1: JOINS del SQL



Fuente: Castro Calderón, 2022, https://www.linkedin.com/pulse/aprende-usar-los-join-en-sql-parte-i-luis-felipe-castro-calder%C3%B3n/?trk=pulse-article_more-articles_related-content-card&originalSubdomain=es.

Selección de campos de forma condicional

Cuando definimos qué campos queremos recuperar, podemos elegir condiciones que permitan recuperar un campo u otro. Para ello, usamos la sintaxis con las palabras **CASE** y **WHEN**.



Veamos la sintaxis de la consulta con campos condicionales:

```
SELECT
  CASE
    WHEN condicion1 THEN campo1
    WHEN condicion2 THEN campo2
    WHEN condicionN THEN campoN
    ELSE campoY
  END,
  campo3, campo4 ...
FROM
  nombre_tabla1, nombre_tabla2, ...
-- CONDICION
WHERE
  ...
```

En la sentencia anterior también es posible usar la relación de tablas con JOIN y, en la parte de condición, si empleamos el WHERE se siguen las mismas reglas que hemos visto anteriormente con operadores lógicos.

Consultas y subconsultas

Una subconsulta (también conocida como SUBQUERY o SUBSELECT) es una sentencia SELECT dentro de otra sentencia SQL. Esto nos permite realizar consultas que, de otro modo, serían extremadamente complicadas o imposibles de realizar.

```
SELECT
-- CAMPOS
  campo1, campo2, ...
FROM
  tabla AS T
WHERE
  campo1 IN
  (
    SELECT
      campo1
    FROM
      tabla2 AS T2
    WHERE
      T.id = T2.id
  )
```



En el ejemplo anterior, tenemos una primera declaración SELECT que realiza un filtro a través del operador lógico **IN** dentro de otro SELECT; es decir, una consulta dentro del resultado de otra consulta.

Veamos algunas maneras de usar subconsultas:

- Subconsulta como una nueva columna de consulta (SELECCIONAR COMO CAMPO).
- Subconsulta como filtro de consulta (mediante **IN**, **EXISTS** u operadores lógicos de comparación).
- Subconsulta como fuente de datos de una consulta principal (SELECT FROM SELECT).

Subconsulta como una nueva columna de consulta

Una de las formas posibles de realizar una subconsulta es convertir el resultado de otra consulta en una columna dentro de su consulta principal.

Veamos la sintaxis necesaria para realizar esta operación:

```
SELECT
    campo1,
    (SELECT
        COUNT(campo2)
    FROM
        tabla_relacionada
    WHERE
        tabla_relacionada.id = tabla.id ) AS cantidad
FROM
    tabla
GROUP BY
    tabla.id
```

Subconsulta como filtro de una nueva consulta

Otro ejemplo de uso de subconsultas es filtrar los resultados de otras consultas. Para este modelo podemos utilizar las cláusulas **IN**, **EXISTS** u operadores de comparación como **=**, **>=**, **<=**, entre otros.



La sintaxis funciona de la siguiente manera:

```
SELECT
    campo1, campo2, ...
FROM
    tabla
WHERE
    tabla.campo1 IN
    (
        SELECT
            campo2
        FROM
            tabla2
        WHERE
            tabla.campo1 = tabla2.campo2
    );
```

Procesos programados

En SQL podemos realizar operaciones programadas a partir de eventos o a partir de la solicitud de ejecución. Estas operaciones pueden ser complejas y son capaces de realizar operaciones sobre tablas, veamos sus aplicaciones.

Triggers

Un *trigger* es una clase especial de procedimiento almacenado que se ejecuta automáticamente cuando se produce un evento en el servidor de bases de datos. Los *trigger* se ejecutan cuando un usuario intenta modificar datos mediante un evento de lenguaje de manipulación de datos (DML). Los eventos DML son instrucciones [INSERT](#), [UPDATE](#) o [DELETE](#) de una tabla o vista. Estos *triggers* se activan cuando se desencadena cualquier evento válido, con independencia de que las filas de la tabla se vean o no afectadas.



(Estrada Web Group, s.f., <https://estradawebgroup.com/Post/Que-es-un-trigger-o-desencadenador-y-como-crearlo-/1032>)

La sintaxis varía entre los DBMS, pero se sigue una estructura común:

```
CREATE TRIGGER nombre_trigger
AFTER INSERT/UPDATE/DELETE ON tabla
FOR EACH ROW
BEGIN
-- CODIGO A EJECUTARSE
END;
```

¿Para qué sirve un *trigger*?

La principal función de los *triggers* es contribuir a **mejorar la gestión de la base de datos**. Gracias a ellos, muchas operaciones se pueden realizar de forma automática, sin necesidad de intervención humana, lo que permite ahorrar mucho tiempo.

Otra de sus funciones es **aumentar la seguridad e integridad de la información**. Esto lo consiguen gracias a la programación de restricciones o requerimientos de verificación que permiten minimizar los errores y sincronizar la información.

Por otra parte, entre sus principales ventajas es que todas estas funciones se pueden realizar desde la propia base de datos, es decir, no es necesario recurrir a lenguajes externos de programación. (Ayudaley, s.f., <https://ayudaleyprotecciondatos.es/bases-de-datos/trigger/>)



Store procedures

Un *store procedure* o procedimiento almacenado, es un conjunto de comandos SQL que se pueden ejecutar a la vez, como en una función. Almacena tareas repetitivas y acepta parámetros de entrada para que la tarea se realice de acuerdo a la necesidad individual.

Beneficios de usar procedimientos almacenados

Un procedimiento almacenado proporciona una importante capa de seguridad entre la interfaz de usuario y la base de datos. Admite seguridad a través de controles de acceso a datos porque los usuarios finales pueden ingresar o cambiar datos, pero no escribir procedimientos. Un procedimiento almacenado preserva la integridad de los datos porque la información se ingresa de manera consistente. Mejora la productividad porque las declaraciones en un procedimiento almacenado solo deben escribirse una vez.

...

El uso de procedimientos almacenados puede reducir el tráfico de red entre clientes y servidores, porque los comandos se ejecutan como un solo lote de código. Esto significa que solo la llamada para ejecutar el procedimiento se envía a través de una red, en lugar de que cada línea de código se envíe individualmente. (Equipo técnico Krypton, s.f., <https://kryptonsolid.com/que-es-un-procedimiento-almacenado/>)



La sintaxis básica de la creación de un procedimiento almacenado es:

```
CREATE PROCEDURE nombre_procedimiento  
AS  
acciones  
GO;
```

Luego, para ejecutarlo hacemos lo siguiente:

```
EXEC nombre_procedimiento;
```



Referencias

Ayudaley (s.f.). *¿Qué es un trigger en una base de datos?* Ayudaley.

<https://ayudaleyprotecciondatos.es/bases-de-datos/trigger/>.

Castro Calderón, L. (2022). *Aprende a usar los JOIN en SQL - Parte I*. LinkedIn.
https://www.linkedin.com/pulse/aprende-usar-los-join-en-sql-parte-i-luis-felipe-castro-calder%C3%B3n/?trk=pulse-article_more-articles_related-content-card&originalSubdomain=es.

Equipo técnico Krypton (s.f.). *¿Qué es un procedimiento almacenado?* Krypton Solid. <https://kryptonsolid.com/que-es-un-procedimiento-almacenado/>.

Infase (6 de septiembre de 2017). *Motores de Bases de Datos*. Infase.
<https://infase.net/motores-de-bases-de-datos/>.

Strada Web Group (s.f.). *¿Qué es un trigger o desencadenador y cómo crearlo?* Estrada web group. <https://estradawebgroup.com/Post/Que-es-un-trigger-o-desencadenador-y-como-crearlo-/1032>.